

BranchLedger System Documentation

Problem definition analysis design and assurance

Version 5.0

4 October 2026

Proposed system specification

This document defines the problem, requirements and proposed delivery procedure for BranchLedger. It retains the detailed accounting, synchronisation, security and assurance specifications. Implementation and actual system test results remain to be verified.

Table of Contents

1 Problem Definition.....	2
2 Investigation of the Existing System.....	3
3 Feasibility Study.....	4
4 Requirements Analysis.....	5
5 System Design.....	8
6 System Implementation.....	15
7 Testing Validation and Verification.....	16
8 Deployment and Changeover.....	24
9 System Operation and Maintenance.....	25
10 Evaluation and Future Development.....	26
References.....	26
Appendices and companion documents.....	26

1 Problem Definition

1.1 Background

Businesses with a main branch and subbranches need accurate sales, expenses, stock and financial information. Separate records and unreliable connectivity can delay consolidation and management decisions.

1.2 Problem statement

The problem is the difficulty of obtaining accurate, timely and consolidated accounting information from every branch when connectivity is unreliable. Separate records, repeated submissions and inconsistent reporting can introduce duplicates, incomplete information and differences between branch records and company reports.

1.3 Problems to address

Delayed reporting; fragmented records; duplicate synchronisation submissions; inconsistent calculations; interrupted offline work; limited branch visibility; inadequate accountability; and unclear data freshness.

1.4 Project aim

Develop a simple and reliable multi-branch accounting and reporting system that supports offline capture and online synchronisation, allowing authorised management to monitor the entire business.

1.5 Specific objectives

Capture sales, purchases, expenses and stock movements; support authorised offline operations; synchronise pending transactions safely; prevent duplicate posting; enforce validation and balanced journals; provide consolidated reports; apply role and branch access; preserve audit trails; display freshness; and verify behaviour through documented testing.

1.6 Scope

Company and branch setup, user access, accounting, stock, offline capture, synchronisation, dashboards, reports, audit trails and recovery. Grocery and restaurant specialist workflows form the initial validation scope. Additional categories require separate requirements and validated extensions.

1.7 Stakeholders

Business owner, general manager, branch managers, accounting staff, authorised transaction-entry staff and system administrator.

1.8 Expected benefits

Reduced reporting delays and manual consolidation, improved record consistency, continuity during connectivity interruptions and clearer management visibility.

1.9 Success criteria

Approved requirements and recorded evidence must demonstrate duplicate-free retries, balanced postings, correct consolidation, enforced permissions, recoverable records and explicit unsynchronised-data indicators.

2 Investigation of the Existing System

2.1 Current process and investigation status

Separate branch records and manual consolidation are the working problem assumptions. No completed client interview, observation or document review is claimed. Confirm these assumptions with the intended business before implementation.

2.2 Investigation procedure

Interview owners, managers and accounting staff; observe daily entry and closing; review authorised sample receipts, stock records and reports; assess connectivity and devices; document exceptions and obtain approval of the resulting requirements.

3 Feasibility Study

3.1 Technical feasibility

Assess Go, Laravel Blade, database transactions, offline browser storage, supported devices and network conditions through a prototype. One designated offline stock and close writer per branch is the pilot constraint.

3.2 Economic feasibility

Estimate development, hosting, devices, connectivity, training, backup and support costs against reporting effort and error reduction. A confirmed budget and measured return are not yet available.

3.3 Operational schedule and legal feasibility

Confirm staff readiness, approval responsibilities and a phased pilot schedule. Verify applicable Zimbabwe tax, fiscalisation, privacy and retention obligations separately before production; this specification does not claim legal compliance.

4 Requirements Analysis

4.1 Document control and interpretation

Version 5.0 replaces the earlier overview and safeguards addendum. This specification is for the product owner, developers, accountant, security reviewer and pilot team. Requirements describe intended behaviour; no implemented application has been inspected. Proposed defaults must be confirmed during the pilot.

The product serves one or more branches of a company. The main branch is an ordinary operating branch with an additional organisational label; it is included in the same reporting rules as every sub-branch. Separate legal entities must not be merged as if they were ordinary branches.

Term	Meaning
Company	Customer tenant and financial reporting boundary
Branch	Operating location within that company
Operation	Stable command ID used for retries and duplicate-safe processing
Posting	Authoritative balanced journal committed by Go
Sync	Exchange of operations and authorised changes while online
Projection	Rebuildable summary derived from committed records
Reconciliation	Comparison of independently verified records and balances

Each requirement uses MUST for release obligations and SHOULD for preferred behaviour. A specialist category can appear in setup while its advanced module remains unavailable; the interface must clearly identify available capabilities. No promise is made to satisfy every regulated industry workflow.

4.2 Business configuration and scope boundaries

Setting	Required behaviour	Failure handling
Primary category	Select category and optional secondary activities	Other or mixed remains available
Branch category	Override relevant operational defaults per branch	Changing category does not rewrite history
Module selection	Enable accounting plus selected validated modules	Show prerequisites and unavailable modules
Currencies	Select reporting currency and allowed transaction currencies	Reject unsupported currency or missing rate
Reporting year	Set financial year and first open period	Prevent overlapping or inconsistent periods
Device model	Assign authorised devices and offline writer	Block conflicting stock writers during pilot

Retail presets enable items, receiving, stock counts, returns and expiry where applicable. Restaurant presets enable menu items, recipe versions, ingredient units and waste. Service presets enable service invoices and client balances. Other modules require their own acceptance tests before general availability.

Business type changes only default labels, module visibility and settings. Financial validation remains common. Selecting a category never removes audit history or silently converts inventory into services. The accountant reviews any opening or conversion journal.

4.3 Product purpose and scope

BranchLedger is a configurable accounting and business monitoring platform for businesses in Zimbabwe. Each company selects its primary business category and optional activities during setup. Branches may use different categories under the same company. Common accounting is available across categories; specialist modules require separate implementation and validation.

The first release focuses on accounting, branch control, offline records, daily closing and understandable owner reporting. Grocery and restaurant workflows receive the first specialist validation. Payroll, HR, chat and advanced forecasting are later extensions. Tax submission and fiscalisation are separate integrations whose requirements must be verified before advertising support.

Category	Relevant extension
Retail and wholesale	Products, stock, returns, expiry and cost of sales
Restaurants	Menus, recipes, ingredients, portions and wastage
Services	Service invoices, client debts and project income
Construction	Budgets, materials, subcontractors and progress billing
Manufacturing	Raw materials, production and finished goods
Agriculture	Inputs, crop and livestock activities and sales
Transport	Trips, fuel, vehicle expenses and delivery billing
Accommodation	Bookings, deposits and guest billing
Other or mixed	Products, services and configurable categories

4.4 Users and access

Role	Intended access
Owner	Company-wide reports, setup and authorised approvals
General manager	Assigned branches, comparisons and delegated approvals
Accountant	Ledger, reconciliation, adjustments and period closing
Branch manager	Assigned branch, daily close and local approvals
Branch staff	Permitted entry and own branch tasks
Platform administrator	Service administration with controlled support access

Go checks company, branch and action permissions on every request. A role name alone does not grant unrestricted access. Support access is time limited, approved and audited. A user cannot approve their own request where separation of duties is required.

4.5 Functional requirements

ID	Requirement	Acceptance evidence
FR01	Company and branch setup	Mixed-category branches remain within one company
FR02	Guided transactions and documents	Valid entries save with unique IDs and source documents
FR03	Double-entry accounting	Every posted journal balances and retains its source
FR04	Offline entry and online sync	Interrupted and repeated uploads create one record

ID	Requirement	Acceptance evidence
FR05	Daily close and approvals	Cash differences and approvals remain traceable
FR06	Cash and bank reconciliation	Matches and unresolved differences are reported
FR07	Branch transfers	Sender and receiver confirmations are linked
FR08	Inventory and cost of sales	Quantity and valuation reconcile with ledger
FR09	Dashboard and reports	Totals link to records and show freshness
FR10	Corrections and period locks	Posted entries survive linked reversals
FR11	Recovery and export	Restored data reconciles and exports remain usable
FR12	Category configuration	Irrelevant tasks are hidden without bypassing access controls

4.6 Detailed permission matrix

Action	Owner	General manager	Accountant	Branch manager	Staff
View company totals	Yes	Delegated	Yes	No	No
View assigned branch	Yes	Yes	Yes	Yes	Yes
Create daily records	Assigned	Assigned	Assigned	Assigned	Assigned
Approve own request	No	No	No	No	No
Approve spending	Limit	Limit	Delegated	Limit	No
Reverse posting	Delegated	Request	Yes	Request	No
Lock periods	Delegated	No	Yes	No	No
Manage users	Yes	Delegated	No	Branch only	No
Export financial data	Yes	Delegated	Yes	Branch only	No
Change chart of accounts	Delegated	No	Yes	No	No

Assigned means both role permission and company/branch membership are required. Delegated means an explicit permission, not an automatic role privilege. Owners who also initiate spending require a second authorised approver where separation of duties applies. Platform support has no routine financial access.

Every route performs server checks even if the button is hidden. Permission changes increment an access version, revoke online credentials where needed and take effect during the next online validation. Offline leases limit disconnected access but cannot provide instantaneous revocation.

5 System Design

5.1 Offline records and synchronisation

Use a cached browser application shell with JavaScript, a service worker and IndexedDB. Laravel Blade renders the initial interface online; it cannot execute PHP or render new server pages without a server connection. Offline forms must already be cached and save through browser code. Initial enrolment requires internet.

Create a UUID for each operation, record its company, branch, device, schema version, actor and local timestamp, and atomically save the record with an outbox item. States are Pending, Syncing, Synced and Failed; business states such as Draft, Awaiting approval and Posted remain separate.

Push bounded batches over HTTPS. The server verifies access, checks the idempotency key and payload hash, validates business rules and commits accepted operations. The same key and same payload return the original result; the same key with different content produces a conflict. Only a durable acknowledgement marks an item Synced.

Pull authorised changes using a server-issued cursor. Apply changes and cursor together locally. Use bounded retries with exponential backoff and jitter. Resume attachments by upload session and checksum. Keep failed operations available for review and never delete them merely because a retry failed.

Independent offline devices cannot guarantee a current shared stock balance. For initial deployments, use one designated offline writer per branch for stock and cash closing. Other devices may read or create restricted independent drafts. A later LAN branch service can coordinate multiple writers. Central validation queues stock conflicts for review rather than silently rewriting records.

Offline access uses a configurable, bounded authorisation lease. Once it expires, require reconnection before new entry. Revocation cannot instantly reach a disconnected device. The server rechecks current permissions on reconnect and quarantines rejected records for authorised review.

5.2 Screen catalogue and navigation

Screen	Primary actions	Required state
Manager overview	Filter, compare and drill down	Currency, freshness and incomplete branches
Branch overview	Daily totals, quick entry and sync	Assigned branch and operating date
Transactions	Search, create, view, reverse	Posting and sync states separately
Customers and suppliers	Create, view balances and allocate settlements	Company scope and ageing date
Purchases and receiving	Enter invoice and confirm quantities	Invoice duplicate warnings
Stock	Count, adjust, waste and transfer	Valuation and unresolved conflicts
Approvals	Inspect, decide and escalate	Limit, decision and reason
Daily close	Count cash and submit	Expected cash and discrepancy
Reconciliation	Match statement items and resolve	Matched and outstanding entries
Reports	Filter, export and drill down	Report basis and generation time
Sync centre	Retry, inspect failures and back up	Last successful push and pull
Settings	Company, accounts, devices and limits	Authorised changes with audit

The role-specific menu shows only permitted tasks. Every entry screen displays branch, currency and offline state. Loading states indicate progress; empty states distinguish no matching data from delayed sync or unavailable access. Errors name the field and safe next step without exposing secrets.

5.3 Transaction fields and validation

Field	Type and rule	Example
operation_id	UUID generated once; immutable	Unique device command
branch_id	UUID; authorised company branch	Main branch
document_type	Allowlisted enum	sale, purchase, expense, receipt
document_date	Local operating date within permitted period	2026-10-04
currency_code	Configured code with scale	Demonstration currency
exchange_rate	Positive exact decimal; reporting units per original unit	1.000000
counterparty_id	Company-scoped customer or supplier	Customer C001
lines	One or more validated lines	Item, quantity, unit price
quantity	Positive decimal with configured unit precision	2.000
unit_price	Non-negative exact decimal	25.00
discount	Authorised amount or percentage	5.00
tax_code	Configured reviewed rule	Exempt or applicable code
payment_account_id	Eligible cash, bank or mobile account	Branch cash
source_reference	Bounded text; duplicate checks	Supplier INV-102
attachment_ids	Company-owned verified objects	Receipt scan
expected_version	Integer for optimistic concurrency	3

Proposed text limits: company name 160 characters, source reference 100, explanation 2,000 and external description 500. Validate Unicode without destructive blanket sanitisation. Dates use an explicit company timezone; server receipt time is separate from operating date. Quantity, price and currency precision must be declared per field.

Line net = quantity times unit price minus authorised discount. Apply the configured tax basis and consistent rounding. The server calculates authoritative totals and rejects client mismatches. Offline totals are provisional until posted; stale price-list versions require review where the policy disallows them.

5.4 Sales receipts expenses and purchase workflows

Sale and customer receipt

1. Select branch, date and customer or cash sale. Add items or services and permitted discounts.
2. Calculate net, tax and total using the configured rules. Select immediate payment or credit terms.
3. Save the complete document and outbox locally. Display its stable reference.
4. On sync, Go validates scope, period, prices, stock policy and posting rules. Commit the journal and stock movement together.
5. For credit sales, allocate subsequent customer receipts to invoices. Reject settlement beyond the outstanding balance unless an explicit customer advance is recorded.

Purchase and supplier payment

1. Select supplier and invoice reference. Check possible duplicates before recording.
2. Enter received items, unit costs, tax and payment terms. Partial receiving must retain the unreceived quantity.

3. Confirm receiving and approval requirements. Separate goods receipt from supplier payment where applicable.
4. Post inventory or expense and the payable or payment account. Link later payments to supplier invoices.
5. Keep supplier credits, returns and advances distinct; never silently reduce an invoice by editing history.

Expense and supporting evidence

An expense requires category, payment or payable account, date, currency, explanation and evidence according to policy. Above-limit requests enter Awaiting approval. Rejected requests retain their reason and do not post. Staff may correct and resubmit an unposted request under a new revision.

5.5 Posting catalogue and worked examples

Examples below use fictional amounts in one reporting currency and exclude tax. They illustrate accounting design, not tax advice. An accountant must review the complete rules before implementation.

Event	Debit	Credit	Amount
Cash sale	Cash	Sales revenue	100
Goods consumed by sale	Cost of sales	Inventory	60
Credit sale	Accounts receivable	Sales revenue	200
Customer receipt	Bank	Accounts receivable	200
Goods bought on credit	Inventory	Accounts payable	150
Supplier payment	Accounts payable	Bank	150
Cash operating expense	Operating expense	Cash	20
Equipment purchase	Fixed asset	Bank or payable	500
Stock waste	Stock loss expense	Inventory	15

Cash sale example: sales 100 less cost 60 gives gross profit 40. After cash expense 20, net profit from these activities is 20. Cash increase is 80, because the inventory cost may have been paid earlier. This demonstrates why cash and profit must be presented separately.

Credit sale example: sale 200 records revenue and a customer debt. Receipt 200 later reduces the debt and increases bank; it does not record another sale. The same rule prevents supplier payments from duplicating purchase expense.

Use a reviewed moving weighted-average inventory method for the initial grocery module. Ten units at 5 and ten at 7 give a cost pool of 120 for 20 units, average 6. Selling four units produces cost of sales 24 and remaining valuation 96. Preserve valuation precision and round at the declared posting boundary.

Foreign currency example: original amount 100 at 2 reporting units per original unit posts 200 reporting units. If settlement occurs at 2.1, bank is 210 and the additional 10 is a separately reviewed exchange difference. Never mix currencies by adding original amounts directly.

Purchase taxes, output taxes, payroll, depreciation, accruals, refunds, tax-inclusive pricing and revaluation require individual approved posting templates. The system must prevent use of unverified templates in production.

5.6 Daily closing approvals and period controls

Expected physical cash per branch and currency equals approved opening float plus cash receipts and cash transfers received, less cash payments, cash refunds and transfers dispatched. Bank and mobile-money receipts do not form part of physical cash. Counted cash minus expected cash gives the cash difference.

State	Allowed transition	Required evidence
Open	Submit close	Counts, movements and explanation
Submitted	Approve or return	Authorised independent review
Returned	Correct and resubmit	Reason and retained revision
Approved	Reopen by authorised correction	Reason and linked amendment
Period open	Lock	Trial balance and reconciliation review
Period locked	Controlled reopen or next-period adjustment	Accountant approval and audit

Approval thresholds have currency and effective dates. Compare requests using the recorded rate where thresholds use reporting currency. Missing rates block approval. Prevent self-approval and check current record version to avoid approving an obsolete request.

A daily close summarises activity and must not itself recognise sales again. A final central close requires synchronised underlying records. Offline closes are provisional. Outstanding drafts, unresolved conflicts and missing branches are shown before period locking.

5.7 Inventory restaurant recipes and transfers

Area	Required rule	Exception
Receiving	Increase quantity and cost only for confirmed receipt	Partial receipt remains open
Sale	Reduce quantity and post cost of sales	Insufficient stock becomes review item
Waste	Record reason, quantity and approved loss	High losses require manager approval
Stock count	Compare counted and expected stock at a defined cut-off	In-transit and unsynced records flagged
Recipe	Version ingredient quantities and serving yield	Substitution records require approval
Branch transfer	Sender dispatch and receiver acceptance linked	Shortfall remains unresolved
Units	Use explicit unit conversion and precision	Incompatible units rejected

Restaurant example: one portion uses 0.20 kg rice and 0.15 kg chicken. Ten portions imply 2 kg rice and 1.5 kg chicken under that recipe version. At ingredient costs 1.50/kg and 4/kg, expected cost is 9. Physical usage can differ due to waste, yield or substitution and requires count-based reconciliation.

Never calculate restaurant profit from sales minus general expenses while omitting ingredient costs. Recipe-based usage is an expected movement; establish the approved reconciliation method so stock-count adjustments do not duplicate consumption.

Money transfer example: sending branch credits cash and debits transfer clearing 100. Receiving branch debits cash and credits clearing 100 on receipt. Consolidated clearing is zero after full receipt. Before receipt, the clearing amount represents in-transit funds and remains visible.

5.8 Reporting definitions and manager chart rules

Metric	Calculation and basis	Display rule
Revenue	Posted revenue credits less reversals and returns	Exclude tax collected as a liability
Gross profit	Revenue minus cost of sales	Show cost completeness

Metric	Calculation and basis	Display rule
Net profit	Revenue minus recognised costs and expenses plus other income	Defined accounting basis
Cash balance	Opening plus posted receipts less posted payments	Separate currency and account
Receivables	Customer invoices less allocated receipts and credits	Age by due date
Payables	Supplier invoices less allocated payments and credits	Age by due date
Branch sales share	Branch revenue / positive company revenue	Suppress invalid percentage cases
Trend change	Current comparable period minus prior period	No percentage when prior value is zero
Sync freshness	Last successful acknowledgement and pull per branch	Flag stale or never-synced branch

Reports must state branch scope, period, currency, posted/provisional basis, generation time and projection watermark. Balance sheet is as of a date; profit and loss covers a period; ageing is as of a date. Ledger drill-down preserves the exact filters and accounting basis.

Use bars for branch comparisons and negative values, lines for time trends, and pie or doughnut charts for a small positive composition. Label amounts and percentages where useful, avoid 3D effects, and offer a table view. Colour must be accompanied by text. A stale branch gets a visible badge and company results become incomplete.

Cash flow requires classified cash movements and a reviewed method, not a simple profit graph. Trial balance total debits must equal total credits. Balance sheet assets must equal liabilities plus equity. Report tests reconcile to seeded journals and posted source records.

5.9 Data lifecycle imports corrections and exports

Each record retains company, branch, author, server times, operating date, currency, revision, source and status. Draft changes are versioned. Posted transactions cannot be deleted. Reversal records reference the original and retain reason, approver and posting period.

Import stages are Uploaded, Validated, Previewed, Approved, Committed and Rejected. Store source checksum, mapping version and row keys. Re-running the same batch returns its original result. A new batch with the same invoice references is flagged before posting. Partial commit policy must be explicit; the initial design uses a reviewed batch with per-document atomic commits and a complete result manifest.

Exports run in background, carry scope and watermark, and are available only to authorised users through expiring links. Neutralise spreadsheet formula injection in user-entered text. Set retention for temporary exports separately from accounting records. Account cancellation follows approved retention and data-export procedures.

5.10 Dashboards reports and analysis

The manager overview shows revenue, expenses, gross profit, net profit, cash and bank balances, customer debt, supplier payables, pending approvals and branch freshness. Clickable bars compare branches; line charts show trends; a small doughnut shows positive expense categories. Negative results use bar or line charts.

Filters include company or branch, date range, business activity and reporting currency. Labels use plain language and visible amounts. Use comparable periods for percentage changes. If the prior value is zero, show the absolute difference rather than a misleading percentage. Flag missing branches and provisional periods.

Provide daily branch summary, profit and loss, balance sheet, cash flow, trial balance, general ledger, ageing, cash reconciliation, transfers, inventory valuation and audit history. Financial totals come from posted journals. Refresh summaries after committed events and provide report generation time and reconciliation status.

Alerts use explainable rules: debt beyond due date, cash difference, expense above configured threshold, overdue branch sync, low stock or unresolved conflict. Each alert names the affected records, reason, severity and next action. AI is optional.

5.11 Document and approval state machines

A source document moves Draft to Submitted, then either Awaiting approval, Ready to post or Rejected. Go alone moves Ready to post to Posted after an atomic commit. Validation failure retains the submitted revision and structured errors. Rejection requires a reason. Resubmission creates a new revision. Posted documents remain immutable and a linked reversal or adjustment records corrections. A reversal may reverse quantities and financial effects only once under the approved workflow.

Sync state is independent: Pending becomes Syncing, then Synced after durable acknowledgement. Transient network failures return to Pending with retry time. Permanent business failures become Failed for review. Synced can describe a centrally stored document awaiting approval, so screens always show both states. Cancellation of an unposted draft preserves the audit event and cancels its unprocessed command safely. A sent command with unknown outcome must first resolve its operation status.

5.12 Financial posting examples and control totals

The examples are fictional, tax-exclusive and in one reporting currency. They define test fixtures, not tax rates. Each journal has equal total debits and credits. A source correction links both journals and maintains its explanation.

Event	Debit	Credit	Expected control
Owner capital 1,000	Bank 1,000	Capital 1,000	Balance sheet balances
Inventory credit purchase 300	Inventory 300	Supplier payable 300	Stock schedule equals inventory control
Cash goods sale 200 at cost 120	Cash 200 and cost of sales 120	Revenue 200 and inventory 120	Gross profit 80
Rent cash expense 30	Rent 30	Cash 30	Operating profit 50
Supplier payment 100	Supplier payable 100	Bank 100	Remaining supplier payable 200
Transfer dispatch 40	Funds in transit 40	Sender cash 40	No revenue
Transfer receipt 40	Receiver cash 40	Funds in transit 40	Transit clears
Cash refund 20 with recovered cost 12	Sales returns 20 and inventory 12	Cash 20 and cost of sales 12	Original sale linked

For the first five rows only, closing bank is 900, cash is 170 and inventory is 180. Assets total 1,250. Supplier payable is 200, capital is 1,000 and current profit is 50. Liabilities plus equity total 1,250. Trial balance and balance sheet must reproduce these totals. Transfers and refunds are separate additional scenarios.

5.13 Chart and report calculation specification

Net sales equals posted sales less sales returns and permitted discounts, excluding taxes collected for another party. Gross profit equals net sales less posted cost of sales. Operating profit equals gross profit plus operating income less operating expenses. Account mappings define which accounts participate, and all mappings are versioned. Cash balance is the ledger balance of selected cash accounts, not revenue.

Branch contribution equals branch net sales divided by company net sales only when the denominator is positive and categories are nonnegative. Otherwise use labelled bars. Period change equals current minus previous divided by the absolute previous value, with zero prior values shown as Not comparable. Comparing incomplete periods requires a visible warning. Ageing uses invoice due date, allocates posted credits and settlements, and groups outstanding amounts into current, 1 to 30, 31 to 60, 61 to 90 and over 90 days.

Profit and loss uses period movements. Balance sheet uses cumulative balances at the selected date, including current earnings. Trial balance lists debit and credit account balances on a consistent basis. Cash flow requires a reviewed account and transaction classification, including noncash exclusions, and must reconcile opening and closing cash equivalents. Stock valuation equals remaining quantity valued by the approved costing policy. Dashboard exports carry company, branches, period, reporting currency, posting basis, generated time and branch freshness.

6 System Implementation

6.1 Implementation procedure

Build the Go authoritative transaction and synchronisation services, database migrations and Laravel Blade session gateway and views. Implement browser JavaScript, service worker and IndexedDB for supported offline work. Version API contracts, use isolated environments and complete each module acceptance contract before integration.

6.2 Detailed module acceptance contracts

The following contracts extend the preceding module catalogue. All acceptance cases require recorded evidence. A category preset changes defaults and available screens, while the same company boundary, posting engine and audit controls apply to every enabled module.

Module	Fields	Validation	Acceptance
Company setup	Company name, branch code, category, timezone, financial year, reporting currency, enabled modules	Unique branch code per company; at least one main branch; no overlapping periods	Opening schedules reconcile to control accounts before Ready
Sales	Customer, operating date, line item, quantity, unit, price version, discount, tax code, payment allocation	Quantity positive; permitted discount; payment allocations equal amount received	100 cash sale posts once and links stock cost
Purchases	Supplier, invoice number, invoice date, receiving reference, quantity, cost, payable account	Duplicate supplier invoice flags; received quantity cannot exceed authorised quantity without approval	Credit receipt 50 increases stock 50 and payable 50
Expenses	Payee, category, currency, amount, payment account, explanation, evidence	Positive amount; evidence policy; approval currency and effective threshold	Unapproved expense produces no journal
Customer receipt	Customer, invoices, allocations, payment account, reference	Allocation cannot exceed outstanding amount; advances explicitly separated	Receipt settles debt without new revenue
Supplier settlement	Supplier, invoices, allocations, currency, paying account	Balance checked under lock; approval before posting	Two concurrent full settlements cannot overpay silently
Daily close	Branch, day, currency, opening float, expected cash, count, explanation	One active close revision per branch day currency; independent approval	Shortage remains visible until approved resolution
Stock count	Count session, cut-off, item, unit, physical quantity, reason	Freeze or snapshot cut-off; pending movements identified	Approved difference creates quantity and valuation movement
Transfers	Sender, receiver, amount or quantities, dispatch, receipt, shortfall	Same company; different branches; receipt limited to remaining balance	Partial receipt leaves residual in transit
Reports	Scope, period, currency, posting basis, freshness cutoff	Authorised branches only; consistent conversion basis	Drill-down sum equals displayed total
Imports	Batch ID, source checksum, mapping, cut-off, rows	Preview before commit; immutable source identity; row errors retained	Same committed batch retry returns original outcome

7 Testing Validation and Verification

7.1 Acceptance tests and traceability

Test ID	Scenario	Expected evidence
AT01	Offline sale then reconnect	One source, balanced journal, one stock movement
AT02	Acknowledgement lost then retry	Same server record ID and one posting
AT03	Same operation ID different body	Conflict with no second posting
AT04	Cross-company branch ID	Access denied and audited
AT05	Stale version approval	Conflict and refreshed request
AT06	Credit invoice then receipt	Debt settles without duplicate revenue
AT07	Closed-period edit	Denied or authorised reversal workflow
AT08	Partially received transfer	In-transit amount remains visible
AT09	Recipe change	Historical sale retains old recipe version
AT10	Dashboard stale branch	Incomplete total and last sync badge
AT11	Currency conversion	Original rate and exact converted amounts retained
AT12	Restore and pending local replay	Known balances recovered without duplicates
AT13	Update with pending IndexedDB records	Pending operations and IDs preserved
AT14	Forged webhook	No payment credit or subscription activation
AT15	Reset and session revocation	Expired reset and prior online session rejected

FR01 through FR12 in the overview map to the detailed modules above. AT01-03 verify FR04; AT04 verifies FR01-02 access; AT05 and AT07 verify FR05 and FR10; AT06 and AT11 verify FR03; AT08 verifies FR07; AT09 verifies FR08; AT10 verifies FR09; AT12-13 verify FR11; AT14-15 verify security and integration requirements. Store fixtures, actual outputs and reviewer decisions.

7.2 Launch controls with verification evidence

The following 50 controls are integrated release requirements, not a separate claim that defects have been removed. Payment and AI controls are conditional on enabled integrations. Every applicable item needs an owner, actual test evidence and reviewer sign-off. All implementation states remain Planned until the team records results.

ID	Control and required behaviour	Acceptance evidence
1	Request and login throttling Apply endpoint, user, device and IP limits.	Burst tests return 429 and retain pending records.
2	API quotas Set per-company limits and bounded sync batches.	Oversized usage fails predictably without partial postings.
3	Paid service spending Set provider budgets and application hard caps.	At the cap, optional paid calls stop and accounting continues.
4	Error handling Return safe messages and stable error codes.	Unexpected errors reveal no secrets or stack traces.

ID	Control and required behaviour	Acceptance evidence
5	Loading states Show progress and prevent repeated submission.	Slow requests show progress without blocking recovery.
6	Empty states Explain no data, no access and no matching results separately.	An empty dashboard identifies filters and freshness.
7	Failed requests Retain data and offer safe retry or escalation.	Connection failures do not erase a saved operation.
8	Timeouts Set request deadlines and cancellation propagation.	Timeout retry uses the original operation ID.
9	Duplicate subscriptions Use unique company entitlement and billing-operation keys.	Concurrent activations create one entitlement.
10	Duplicate payments Use stable payment keys and unique provider references.	Repeated callbacks produce one financial posting.
11	Query efficiency Profile reports and remove unnecessary repeated queries.	Representative load meets agreed latency targets.
12	Database indexes Index company, branch, date and frequent lookup fields.	Query plans use appropriate indexes without tenant leakage.
13	Pagination Bound page sizes and use stable cursor ordering.	Large lists return bounded pages with correct access scope.
14	Compression Compress text and eligible exports without altering source evidence.	Checksums and readability survive compression.
15	Upload sizes Enforce attachment and company-storage limits.	Oversized files fail before unbounded buffering.
16	Upload types Allowlist extensions and verify MIME and file content.	Disguised executables fail and files cannot execute.
17	Caching Scope keys by company, permissions and report filters.	No user receives another company cached data.
18	Uptime monitoring Monitor live service readiness and alert routing.	A simulated outage triggers an actionable alert.
19	Application logging Record request IDs and safe diagnostics.	Errors are traceable without passwords or tokens.
20	Input validation Validate schema, lengths, enums and business relationships.	Malformed and cross-company references fail.
21	Password hashing Use maintained bcrypt or Argon2id implementations.	No plaintext password reaches database or logs.
22	CORS Prefer same origin; allow only required explicit origins.	Unapproved cross-origin browser requests fail.

ID	Control and required behaviour	Acceptance evidence
23	SQL injection Use parameterised queries and allowlisted dynamic identifiers.	Injection inputs cannot change query structure.
24	Sessions Rotate on login and invalidate sessions after password change.	Old sessions and refresh credentials fail online.
25	Dependencies Scan Go, Composer and browser packages in CI.	Unresolved critical or high findings block release unless formally reviewed.
26	HSTS Enable after validating HTTPS on the affected hosts.	HTTPS responses carry reviewed HSTS policy.
27	CSRF Require tokens on cookie-authenticated state-changing routes.	Missing or invalid token fails without posting.
28	Reset expiry Use short-lived single-use reset tokens stored as hashes.	Expired and reused tokens fail.
29	User enumeration Use generic login and reset responses with comparable behaviour.	Existing and absent accounts do not reveal identity.
30	Payment webhooks Verify provider signatures, timestamps and replay protection.	Invalid signature, wrong amount and replay cannot credit accounts.
31	Server prices Derive subscription prices and permitted sale prices on the server.	Tampered totals or unauthorised discounts fail.
32	AI injection safeguards If enabled, constrain tools and treat retrieved content as untrusted.	Adversarial prompts cannot access other companies or post journals.
33	AI quotas If enabled, cap tokens, calls and cost by company.	Usage above the cap fails safely with a clear message.
34	Request body bounds Enforce byte, array, nesting and decompression limits.	Large or deeply nested requests fail early.
35	Data protection Minimise data, encrypt where appropriate and enforce retention.	Export, deletion and access procedures follow approved policy.
36	Directory listing Disable listing and deny private file paths.	Directory requests do not expose files.
37	Admin routes Authenticate and authorise admin actions; remove unused routes.	Knowing an admin URL grants no access.
38	Security events Audit login, access denials, role changes and exports.	Events identify actor, company and server time without secrets.
39	Cookies and database privilege Use Secure, HttpOnly and appropriate SameSite; restrict DB roles.	Cookies and credentials meet reviewed least-privilege policy.
40	Genuine reviews Publish only authorised, real customer feedback.	Every testimonial has a verifiable source and permission.

ID	Control and required behaviour	Acceptance evidence
41	Logo Use consistent original branding; text logos remain acceptable.	Logo is readable and used consistently.
42	Favicon Provide browser and installed-app icons.	Icons render at supported sizes.
43	Writing Review labels, spelling and punctuation consistently.	UI wording is clear; punctuation style is not a security control.
44	Colours Choose an accessible consistent palette.	Text contrast and status labels remain readable without colour alone.
45	Domain Use an owned production domain and valid HTTPS.	Domain, certificates and renewal alerts work.
46	Icons Use a consistent labelled icon set.	Icons remain understandable and accessible.
47	Privacy information Publish accurate data-use and contact information.	Page reflects actual collection, retention and sharing.
48	Terms Publish service limits, ownership, cancellation and support terms.	Links work and terms reflect the offered service.
49	Verifiable metrics Label demos and support public claims with evidence.	No fabricated customer counts, uptime or performance results.
50	Temporary account lockout Add progressive delays or temporary locks with safe recovery.	Brute force slows without enabling indefinite attacker lockout.

No launch is approved when accounting, tenant isolation, duplicate-safe synchronisation, restore or payment verification has an unresolved critical failure. Presentation choices are reviewed for clarity and accessibility. Text logos or punctuation alone are not vulnerabilities.

7.3 Executable pilot scenarios and release evidence

Use a seeded demo company with two branches, a grocery profile and a restaurant profile. Record exact input, expected journal, actual output, version, tester and evidence link. Every critical accounting, isolation, duplicate and recovery case must pass before live records are accepted.

Test	Action	Pass condition
T01	Replay one sale command 20 times	Exactly one journal and stock movement; identical stored result
T02	Drop network after server commit before reply	Retry returns original result and local outbox closes
T03	Reuse operation ID with changed amount	409 conflict; no second journal
T04	Submit branch ID from another company	403 or scoped not-found; no data disclosure
T05	Post unbalanced journal	Reject entire transaction; no source marked Posted
T06	Two receipts settle one invoice concurrently	Total allocation never exceeds allowed balance
T07	Close cash with count 315 and expected	Shortage 5 retained; no automatic sales

Test	Action	Pass condition
	320	edit
T08	Restore central database before last acknowledged sale	Recovery reconciliation identifies sale and replays original ID
T09	Upgrade browser app with pending operations	IDs, payloads and outbox retained; migrations verified
T10	Expire offline permission lease	Retain pending data; block new entry until reauthentication
T11	Recipe changes after historical sale	Old sale keeps prior recipe version and cost movements
T12	Partial stock transfer receipt	Received and residual in-transit quantities reconcile
T13	Repeat committed spreadsheet import	Original result returned; no duplicate source documents
T14	Dashboard includes stale branch	Visible incompleteness and last update; no implied live total
T15	Check financial fixture in section 5.12	Assets 1,250 equal liabilities and equity 1,250

7.4 Complete assurance framework

This plan covers the agreed common accounting, branch, inventory, restaurant, offline, reporting, security, recovery and market-readiness requirements. It combines inspection, automated tests, controlled fault injection and human acceptance. A business category is supported only for the modules actually implemented and verified. Additional specialist modules require new requirements and cases before release.

All cases in this edition have status Not run. No application source, running system or deployment has been supplied for execution. Document review verifies the design and coverage, not implemented security or accounting correctness. Record actual results separately. Planned, Not run, Pass, Fail, Blocked and Not applicable are distinct statuses. Not applicable requires an approved reason and evidence that the related feature is disabled.

Verification asks whether implementation follows the specification. Validation asks whether the delivered workflows and reports meet business needs. Input validation is the runtime enforcement of data and business rules. Each is necessary; none substitutes for the others.

Unit tests cover exact arithmetic, account mappings, rounding, permissions, recipe conversions and state transitions. Integration tests use real isolated PostgreSQL to verify constraints, locks, transactions, jobs and gateway identity. Contract tests verify request/response schemas and compatible old/new clients. End-to-end tests cover the browser, Laravel gateway, Go API and database with actual user flows.

System testing combines modules. Regression reruns affected and critical behaviours after every change. Smoke tests establish that a build is testable; sanity tests focus on a specific fix. Negative, boundary, concurrency, property-based and fault-injection tests probe invalid values, limits and interrupted execution. Performance covers load, stress, backlog and soak. Security covers threat review, code/dependency checks and authorised penetration tests in isolated environments.

Accountants validate journals, costing, reports and opening balances independently. Staff validate daily workflows. Owners/managers validate visibility and investigations. QA owns case execution and evidence. Engineering fixes defects. Security and operations reviewers validate their areas. The product owner authorises release only against recorded results. Tests that require provider callbacks use sandbox providers, never real charges.

Maintain separate development, automated integration, staging and pilot environments. Use fictional tenants Alpha and Beta, each with at least two branches and users for every role. Beta provides foreign-company IDs for isolation tests. Enable mixed grocery and restaurant branches in Alpha. Never copy production personal or financial data into tests without an approved lawful process.

Fixtures F01: capital 1,000, inventory credit purchase 300, sales 200 at cost 120, rent 30 and supplier payment 100. F02: customer invoice 100, partial settlement 40 and due date yesterday. F03: stock 10 at 5 plus 10 at 7. F04: recipe rice 0.20 kg per portion with effective versions. F05: original currency 100 at rates 2 and 2.1. F06: main and sub-branch transfers with partial receipt.

Record device/browser versions, operating system, local schema, Go/PHP/Laravel/database versions, application commit, migration version, feature flags, network latency/loss and time zone. Simulate offline, slow network, timeout, dropped acknowledgement, low disk, process crash, concurrent requests and central restore. Repeat tests after upgrade with the retained local outbox. Freeze test clocks for date, expiry and ageing boundaries.

7.5 Field and business validation rules

Run each field rule at UI for helpful feedback, at Go for authoritative enforcement and at database where an invariant can safely be expressed. Laravel validates gateway envelope and session/CSRF but does not become a second accounting engine. The authenticated company and branch scope cannot be overridden by submitted IDs.

Boundary method: for each length/size/amount limit test absent, null, wrong type, invalid enum, minimum minus one, exact minimum, exact maximum and maximum plus one where meaningful. Test whitespace, Unicode, decimal precision, exponent/NaN/infinity encodings and malicious nesting. Ensure rejected requests cannot leave partial posting or erase pending local records.

Area	Rule	Negative cases
Identity and scope	Valid UUID; enrolled device; current authorised company/branch	Cross-company ID, inactive branch, revoked device
Dates and period	Declared timezone; valid date; open accounting period	Impossible date, locked period, cut-off boundaries
Money and rate	Exact decimal; configured scale; currency; positive rate	Negative amount, excess precision, zero/missing rate
Quantity and units	Positive sale/purchase quantity; reviewed compatible unit	Zero/negative quantity, incompatible conversion
Price and discount	Authoritative server price/policy; permitted override	Tampered price, stale version, excess discount
Journal	Approved mapping; one side per line; aggregate balance	Both sides populated, unmatched totals, inactive account
Settlement	Valid invoice/counterparty; remaining-balance lock	Foreign invoice, excess/concurrent allocation
Approval	Exact current revision; threshold; independent approver	Self-approval, expired limit, changed revision
Sync	Stable operation ID; canonical hash; schema; bounded batch	Changed body under ID, unknown version, partial batch
Stock and recipe	Approved valuation; source movement; effective recipe	Insufficient stock, past version edit, double waste
Evidence	Allowed type/size; verified content; company-owned object	MIME spoof, unsafe file, checksum mismatch
Imports	Source checksum; mapping; declared partial policy	Repeat commit, invalid row, duplicate supplier invoice
Reports	Authorised scope; consistent currency/basis; freshness	Foreign branch filter, mixed currency, stale checkpoint

7.6 Test traceability and release gate

Each case requires preconditions, exact data, repeatable numbered actions, expected output, actual output, status, tester, execution timestamp and environment/build. Capture UI state, safe API response, source/operation IDs, journal/stock counts and independently calculated expected balances as applicable. Never capture passwords, tokens, reset links or unnecessary personal data.

Defect report fields: defect ID, case and requirement IDs, build, severity, reproducible steps, expected/actual result, safe evidence, affected tenant/branch scope, owner, workaround and retest outcome. Critical means data loss, duplicate financial effects, incorrect material accounting or unauthorised scope access. High means blocked core workflow or significant incorrect reporting. Medium means recoverable limited impact. Low means cosmetic/minor usability.

After a fix, rerun the failed case and related regression. A screenshot alone cannot prove duplicate prevention, journal balance or recovery. Use database and report evidence. Store test evidence immutably under restricted access with reviewed retention. Link every finding to its final retest; do not delete failed evidence.

Entry gate: reviewed specification and mappings, stable candidate build, seeded fixtures, healthy test environment, authorised fault tests, backup of staging and agreed expected results. No performance number is accepted without its device/network/workload definition.

Exit gate: every critical case passes, all mandatory controls have evidence, every enabled financial report reconciles, no unresolved critical or high defect affects safe core use, the full relevant permission matrix passes, restore and pending-record upgrade pass, and pilot users accept daily tasks. Any lower-severity exception needs a named owner, written impact, mitigation and due date. Unimplemented payment or AI features must be disabled and documented as not applicable.

Pilot gate: main branch and sub-branch complete at least five representative trading days including offline entry and recovery drills. Accountant compares receipts, physical cash, bank statements, supplier/customer schedules and inventory to the system. Proposed performance and recovery targets must pass or be explicitly revised before marketing. Re-run smoke checks after deployment and monitor sync backlog before expanding.

Sign-off record: candidate commit and configuration, test execution summary, unresolved exceptions, accountant decision, security decision, operations decision, business acceptance, product owner decision and rollback plan. Sign-off fields remain Not signed until authorised reviewers actually approve.

The companion BranchLedger Testing Validation and Verification contains the complete QA001 to QA062 procedures, preconditions, expected outcomes, evidence and reviewer responsibilities. It forms part of this specification. Every enabled requirement and all 50 launch controls require execution evidence.

Family	Cases	Required result
Setup	QA001, QA002, QA003, QA004	Critical cases pass; evidence retained
Accounting	QA005, QA006, QA007, QA008, QA009, QA010, QA011, QA012, QA013, QA014, QA015, QA016, QA017	Critical cases pass; evidence retained
Stock	QA018, QA019, QA020, QA021, QA022, QA023, QA024	Critical cases pass; evidence retained
Offline	QA025, QA026, QA027, QA028, QA029, QA030, QA031, QA032, QA033, QA034, QA035	Critical cases pass; evidence retained

Family	Cases	Required result
Security	QA036, QA037, QA038, QA039, QA040, QA041, QA042, QA043	Critical cases pass; evidence retained
Payments	QA044, QA045	Critical cases pass; evidence retained
AI optional	QA046	Critical cases pass; evidence retained
Reports	QA047, QA048, QA049	Critical cases pass; evidence retained
Imports	QA050	Critical cases pass; evidence retained
Exports	QA051	Critical cases pass; evidence retained
Performance	QA052, QA053, QA054	Critical cases pass; evidence retained
Recovery	QA055, QA056, QA057, QA058	Critical cases pass; evidence retained
Compatibility	QA059	Critical cases pass; evidence retained
Usability	QA060	Critical cases pass; evidence retained
Release	QA061, QA062	Critical cases pass; evidence retained

8 Deployment and Changeover

8.1 Installation migration and changeover

Configure infrastructure, secrets, access, monitoring and backups. Rehearse migration with reconciled opening balances. Train staff in capture, synchronisation and reversals. Pilot the main branch and a subbranch, review evidence and obtain release sign-off before phased changeover. Maintain a rehearsed rollback procedure.

8.2 Pilot delivery and unresolved decisions

Decision	Proposed default	Confirmation needed
Devices	Modern desktop and Android browsers with installed PWA	Browser and storage testing
Multiple offline writers	One designated stock and close writer per branch	Pilot operating model
Valuation	Moving weighted average	Accountant and customer review
Offline lease	Configurable bounded period	Operational and security risk review
Attachment limits	Configurable per file and company	Network and storage budget
Capacity	Initial test profile of 20 branches and 10000 daily records	Measured benchmark
Performance	Local save p95 under 1s; dashboard under 3s	Agreed device and network
Recovery	Proposed central RPO 15 min and RTO 4h	Backup infrastructure and drills
Subscription plans	Branch and user limits per plan	Commercial pricing
Fiscalisation	Separate verified integration	Current applicable requirements

Pilot one main branch and one sub-branch. Validate full trading days, offline disruptions and accountant-reviewed balances. Measure task completion, time to find a branch issue, sync backlog, correction frequency and support requests. Expand only after repeatable operation and documented launch approval.

Official technology and security references are listed in the architecture document. The supplied user requirements define scope; examples and proposed thresholds are illustrative and must not be advertised as achieved customer metrics.

8.3 Release responsibilities and decisions register

The accountant signs account mappings, valuation, currency treatment, opening balances and report fixtures. Engineering signs sync integrity, migrations and performance evidence. Security review signs company isolation, session controls, uploads and dependency review. The product owner signs daily usability, training, support and truthful presentation. These roles identify approval evidence; they do not claim any sign-off has occurred.

Before implementation, settle supported browsers and devices, offline lease duration, device encryption and backup policy, stock negative-balance policy, rate approval source, retention, recovery objectives, subscription limits and specialist modules. Zimbabwe tax and fiscalisation integrations need separate current verification. Selecting a category never implies legal or operational completeness for that sector.

9 System Operation and Maintenance

9.1 Operating procedure

Monitor failures, sync backlog and data freshness; reconcile daily totals; check backups and rehearse restoration; review access; patch dependencies; and record incidents and controlled releases. RPO, RTO and capacity targets remain proposed until measured.

9.2 Security recovery and performance

Enforce tenant boundaries in queries and database relationships. Require stronger authentication for privileged accounts. Use secure session cookies, CSRF protection, escaped Blade output, server validation, rate limits and least-privilege database credentials. Keep long-lived tokens out of browser localStorage.

Protect branch devices with full-disk encryption, screen locks and controlled accounts. Browser storage is not automatically encrypted; application encryption needs reviewed key storage and recovery. Request persistent browser storage where supported, monitor quota and provide encrypted backup of unsynchronised records. Clearing browser data can destroy pending records.

Back up the central database and attachments, retain recoverable versions and test restore into an isolated environment. Suggested pilot targets are central RPO 15 minutes and RTO 4 hours, subject to hosting validation. These do not cover unsynchronised local records, which need separate protection.

Suggested performance targets, not measured guarantees: local save p95 below 1 second and dashboard load p95 below 3 seconds on agreed hardware and network. Test 20 branches and 10,000 daily company transactions as an initial load profile. Record batch size, retry count, rejection rate, queue age and summary freshness.

9.3 Proposed capacity and operating targets

The initial benchmark is 20 branches, 100 active users, 10,000 company transactions per day and three years of retained transactions. Test a five-times daily burst and a seven-day offline backlog. Proposed p95 targets are local save under 1 second, online dashboard under 3 seconds and normal command acknowledgement under 2 seconds on the agreed reference network. These are acceptance targets to validate, not achieved performance claims.

Proposed defaults are 100 operations per sync batch, 1 MiB JSON request limit, 10 MiB attachment limit and bounded export jobs. Limits are configurable and enforced at proxy and application boundaries. Dashboard summaries carry a projection checkpoint. If workers lag, show the checkpoint and avoid presenting stale summaries as current. Rate limits account for legitimate sync batches and distinguish login, reads, writes, imports and exports.

10 Evaluation and Future Development

10.1 Evaluation and limitations

Compare implemented behaviour with approved objectives and the requirement traceability matrix. Record test evidence, user acceptance, measured performance and remaining defects. Actual system tests are Not run; no achieved reliability or compliance result is asserted.

10.2 Future development

Consider additional business categories, broader offline writer support and optional payments or AI only after validated requirements and safe core operation. Each extension needs its own acceptance and regression evidence.

References

Laravel Blade documentation <https://laravel.com/docs/blade>

Go database transactions <https://go.dev/doc/database/execute-transactions>

MDN IndexedDB https://developer.mozilla.org/en-US/docs/Web/API/IndexedDB_API/Using_IndexedDB

OWASP Authentication Cheat Sheet

https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html

Appendices and companion documents

Detailed diagrams and data dictionary: BranchLedger Architecture. Operating and training instructions: BranchLedger User Manual. Full QA001 to QA062 scenarios, expected results, evidence and sign-off templates: BranchLedger Testing Validation Verification. All remain part of the complete package.